The Multi-Agent Cost Multiplier, Worked Out by Hand

Why a supervisor over W workers costs several times a single agent — and why a hierarchy roughly doubles the tax with every layer you add.

What this document does. It takes the customer-ops supervisor from Module 2.8 — a routing agent dispatching to `billing`, `refund`, `shipping`, and `techsupport` workers — and prices one task end to end. We treat the single-agent ReAct trajectory cost from Module 0.0 as a fixed building block ($C_1 = \$0.0477$) and then add, in dollars, exactly what the supervisor charges on top: the routing calls, each engaged worker's *own* ReAct sub-loop, and the synthesis call. Every number is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent.

Where this sits. This is **Topic 1 of Module 2.8** — the quantitative spine under the module's "5 signals" decision table and its first pitfall, the *cost spiral* ("every turn = supervisor call + worker call ⇒ 3–5× a single agent"). Its companions: Module 0.0 (the single-agent ReAct cost we reuse as C_1) and Module 2.7 (cost engineering — model tiering, prompt caching, step caps, the levers that bend every curve derived here).

Concepts covered, in order: the single-agent cost C_1 as a building block · the supervisor decomposition $C_{\text{route}} + \sum C_{\text{worker}} + C_{\text{synth}}$ · the multiplier C_{multi}/C_1 · supervisor overhead as pure tax · the hierarchical ∼2×-per-layer compounding · the supervisor latency bottleneck $L_{\text{route}} + \max(L_w) + L_{\text{synth}}$ · "6th tool vs. 2nd agent" and the diminishing returns of specialization.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one task, priced from the single-agent block up

A **supervisor** system runs one task like this: the supervisor reads the request and *routes* it to a worker; that worker runs its *own* ReAct loop to completion and reports a result; the supervisor reads the result and either routes to another worker or *synthesises* a final answer. The workers never talk to each other — every hop goes through the supervisor. So one task is a stack of LLM calls: supervisor routing calls, plus one full single-agent trajectory per engaged worker, plus one supervisor synthesis call.

We do not re-derive the worker cost from scratch — Module 0.0 already did. We reuse its result as a sealed building block: a single ReAct agent running a representative 3-step trajectory costs

$$C_1 \;=\; p_{\text{in}}\Big[NB_w + h\tfrac{N(N-1)}{2}\Big] + p_{\text{out}}\,Na \;=\; \$0.0477 \quad (B_w{=}2000,\ h{=}1800,\ a{=}300,\ N{=}3).$$

The quantities. Every supervisor message gets a size in tokens, chosen round so the sums follow by hand. The mechanics are identical at any size — only the constants move.

Quantity	Symbol	Value here
Single-agent trajectory cost (the building block)	C_1	\$0.0477
Supervisor base context per call (router system prompt + worker roster)	B_s	1,000 tokens
Supervisor routing output (a `Route: next`+reason)	a_s	100 tokens
Supervisor synthesis output (the final answer)	a_y	300 tokens
Worker result appended to the shared transcript	r	1,200 tokens
Input price	p_{in}	\$3 / 1M tokens
Output price	p_{out}	\$15 / 1M tokens
Engaged workers on this task	W	1 – 4

Notation. A routing decision before worker k re-reads B_s plus the $(k-1)$ worker results already on the transcript. The synthesis call re-reads B_s plus *all* W results. Costs are tokens × price; we carry four decimals of a dollar throughout.

Intuition

The supervisor does none of the actual work — it only decides who works next and stitches the answers together. Yet it is on every hop. Think of it as a project manager who never writes code but sits in every meeting: each specialist (worker) does a full day's work, and the manager bills a routing meeting before each one plus a wrap-up meeting at the end. The workers are the cost you *wanted*; the manager's meetings are the cost you *added*. This document measures the meetings.

2 Step 1 — the supervisor decomposition

For one task with W engaged workers, the total cost is three named pieces:

$$C_{\text{multi}}(W) \;=\; \underbrace{C_{\text{route}}(W)}_{\text{supervisor tax}} \;+\; \underbrace{\textstyle\sum_{k=1}^{W} C_1}_{\text{the real work}} \;+\; \underbrace{C_{\text{synth}}(W)}_{\text{supervisor tax}}. \tag{1}$$

The middle term is $W\,C_1$ — one full single-agent trajectory per engaged worker, the work you would have paid for anyway. The outer two terms are the supervisor's own LLM calls. The routing term sums one decision per worker, each re-reading the results accrued so far:

$$C_{\text{route}}(W) = \sum_{k=1}^{W} \Big[(B_s + (k-1)\,r)\,p_{\text{in}} + a_s\,p_{\text{out}} \Big], \qquad C_{\text{synth}}(W) = (B_s + W r)\,p_{\text{in}} + a_y\,p_{\text{out}}.$$

Mini-example — this operation in isolation

Take $W = 2$ (route to `billing`, then `refund`, then synthesise). Routing call 1 reads $B_s = 1000$ tokens; routing call 2 reads $B_s + r = 1000 + 1200 = 2200$. So $C_{\text{route}} = (1000+2200)\cdot 3\times 10^{-6} + 2\cdot 100\cdot 15\times 10^{-6} = \$0.0096 + \$0.0030 = \$0.0126$. Synthesis reads $B_s + 2r = 3400$ tokens and writes $a_y = 300$: $C_{\text{synth}} = 3400\cdot 3\times 10^{-6} + 300\cdot 15\times 10^{-6} = \$0.0102 + \$0.0045 = \$0.0147$. The real work is $2\,C_1 = \$0.0954$. Total $C_{\text{multi}}(2) = 0.0126 + 0.0954 + 0.0147 = \$0.1227$. ✓

3 Step 2 — the multiplier table

Working Equation (1) for $W = 1$ to 4 gives the headline figure: how many times a single agent a supervisor system costs.

W	**workers engaged**	C_{route}	$\sum C_{\text{worker}}$	C_{synth}	C_{multi}	**vs.** C_1
1	`billing`	\$0.0045	\$0.0477	\$0.0111	\$0.0633	1.33×
2	+`refund`	\$0.0126	\$0.0954	\$0.0147	\$0.1227	2.57×
3	+`shipping`	\$0.0243	\$0.1431	\$0.0183	\$0.1857	3.89×
4	+`techsupport`	\$0.0396	\$0.1908	\$0.0219	\$0.2523	5.29×

This is the module's "3–5× cost vs. single agent" (Pitfall 1, the cost spiral), now derived rather than asserted. Two facts jump out. First, the multiplier is *super-linear in* W: doubling the workers from 2 to 4 more than doubles the cost (2.57× → 5.29×), because the supervisor's routing and synthesis calls grow as they re-read more accumulated results. Second, even at $W = 1$ — a supervisor that engages a single worker — you already pay 1.33×: a routing call and a synthesis call wrapped around one agent that could have run alone. That 0.33 is pure coordination tax for the privilege of having a supervisor at all.

Intuition

The supervisor overhead is not amortised away by scale — it gets *worse* with scale. Each new worker adds its honest C_1 of real work *and* lengthens every subsequent supervisor call (more results to re-read before routing, more to read at synthesis). The tax rides on top of a transcript that grows exactly like the single-agent history term from Module 0.0 — the same $O(W^2)$-flavoured re-reading, one level up.

4 Step 3 — the overhead is pure tax

Split Equation (1) into "work you wanted" and "work the supervisor added." At $W = 3$:

$$\underbrace{C_{\text{route}} + C_{\text{synth}}}_{\text{supervisor tax}} = 0.0243 + 0.0183 = \boxed{\$0.0426} \qquad \text{on top of} \qquad \underbrace{3\,C_1}_{\text{real work}} = \$0.1431.$$

The tax is \$0.0426 — about **30%** surcharge over the raw worker cost, and it buys no new reasoning: the workers already produced the answers; the supervisor only routed to them and re-summarised them. This is why the module's first instinct is *not* to split: every split you make is a routing-plus-synthesis round-trip you now pay on every task, forever.

Watch out

The tax compounds with the very thing that makes multi-agent attractive — more specialists. A 6-worker org does not pay $6/3 = 2\times$ the tax of a 3-worker org; it pays more, because each routing call re-reads a longer list of prior results before deciding. "Add another specialist" reads as cheap (just one more C_1) but silently re-prices every supervisor call on the task. The worker cost is linear; the coordination cost is not.

5 Step 4 — hierarchy: the tax roughly doubles per layer

A **hierarchical** system stacks supervisors: a top supervisor routes to sub-supervisors, each of which runs its own supervisor loop over its own workers. The module's warning is blunt — "cost compounds quickly." Here is why, isolated. Hold the leaf work fixed at four worker sub-loops ($4\,C_1 = \$0.1908$) and count only the *coordination* calls.

Shape	coord. calls	coord. tax	total
Flat: 1 supervisor over 4 workers	$4{+}1 = 5$	\$0.0615	\$0.2523
Hierarchical: top over 2 subs (2 workers each)	$2(2{+}1) + (2{+}1) = 9$	\$0.0927	\$0.2835

Going from one management layer to two takes the coordination calls from 5 to 9 (1.80×) and the coordination tax from \$0.0615 to \$0.0927 (1.51×) — a *rough doubling* of the supervisor's own cost, on top of the unchanged leaf work. The mechanism is double-reading: each sub-team's results are read once by its own sub-supervisor and *again* by the top supervisor when it routes and synthesises across sub-teams. Every layer you add re-reads everything the layer below already read.

Mini-example — this operation in isolation

Count the flat case by hand. Over 4 workers the supervisor makes 4 routing calls and 1 synthesis call = 5 coordination calls. Now insert a middle layer: 2 sub-supervisors, each managing 2 workers, so each sub does 2 routing +1 synthesis = 3 calls; two subs = 6. The top supervisor then routes to and synthesises over the 2 subs = $2 + 1 = 3$. Total $6 + 3 = 9$. The leaf work — the four C_1 trajectories — never changed; only the management layer multiplied. Three management layers would push it toward $\sim 2^2$ the flat tax. ✓

6 Step 5 — latency: the supervisor is on the critical path

Cost is not the only tax; latency is the other. Routing is *serial by construction* — the supervisor must decide before any worker can start, and must synthesise after they finish. So even when the workers run in parallel, the supervisor brackets them:

$$L_{\text{task}} \;=\; L_{\text{route}} \;+\; \max_{w} L_w \;+\; L_{\text{synth}}. \tag{2}$$

With $L_{\text{route}} = 0.8$s, $L_{\text{synth}} = 1.2$s and three workers taking $\{3, 5, 4\}$s:

$$L_{\text{task}} = 0.8 + \max(3, 5, 4) + 1.2 = 0.8 + 5.0 + 1.2 = \mathbf{7.0}\text{s}.$$

Parallelism helps the workers — 7.0s instead of the $0.8 + 12.0 + 1.2 = 14.0$s you would pay if the workers ran one after another — but it cannot help the supervisor. The $L_{\text{route}} + L_{\text{synth}} = 2.0$s is on the critical path *no matter how fast or parallel the workers are.* A single agent pays neither bracket.

Intuition

Parallelising workers attacks the $\max(L_w)$ term and nothing else. The supervisor's two seconds are a fixed toll at the entrance and exit of the task. As you make workers faster (or more parallel), that fixed toll becomes a *larger share* of the total — the same way fixed costs

dominate once variable costs are squeezed. There is no topology in which the supervisor steps off the critical path; that is what "centralised control" costs.

7 Step 6 — the 6th tool versus the 2nd agent

The module's pragmatic rule is "add tools until 10+ confuse the model; then, and only then, split." The arithmetic shows why splitting is the expensive move. Adding one *tool* to an existing agent adds one schema — a small fixed block of tokens ($T_{\text{schema}} \approx 120$) re-read each step, *with no extra LLM round-trip*. Adding one *agent* adds a full routing-plus-synthesis round-trip and duplicates context. Compare the two moves against the same single agent:

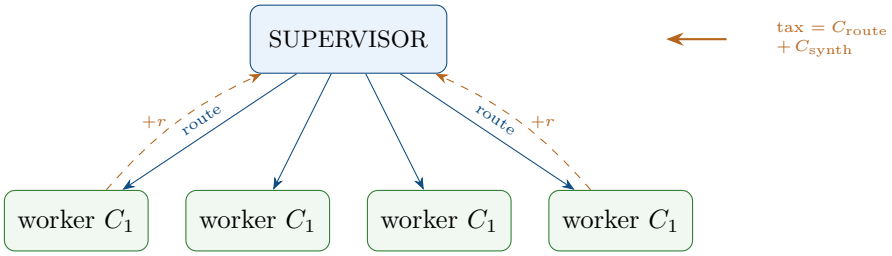
Move	result	added cost
1 agent, 4 tools (baseline C_1)	\$0.0477	—
1 agent, 5 tools	\$0.0488	+\$0.0011
1 agent, 6 tools	\$0.0499	+\$0.0022
1 agent, 7 tools	\$0.0509	+\$0.0032
Split: supervisor + 2 agents	\$0.1227	+\$0.0750
Split: supervisor + 3 agents	\$0.1857	+\$0.1380
Split: supervisor + 4 agents	\$0.2523	+\$0.2046

Going from 4 tools to 6 on a single agent costs +\$0.0022. Splitting that same agent into a supervisor over 2 agents costs +\$0.0750 — about **35**× more for the same broadening of capability. The reason is structural: a tool adds tokens *inside* one round-trip you were already paying for; an agent adds a *new* round-trip (route) and a stitch-back (synthesis), plus the duplicated context each new agent must re-read. Specialisation has sharply diminishing returns — the second agent rarely answers 35× better than a sixth tool would have.

Watch out

The split can still be right — the module lists real reasons (distinct personas, independent failure domains, different model tiers). The point is not "never split" but "splitting is never free, and it is much more expensive than the tool you were tempted to skip." Reach for the 6th tool first; reach for the 2nd agent only when a signal in the module's table genuinely fires.

8 The accounting, drawn



The blue arrows down are routing calls; the dashed orange arrows up carry each worker result r back, which the supervisor re-reads on the *next* routing call and again at synthesis. The green boxes are the only honest work ($W\,C_1$); everything blue and orange is the supervisor tax of Equation (1). Stack a second supervisor above this one and the whole orange-and-blue picture is paid *again* at the higher layer — that is the hierarchy doubling of Step 4.

9 Cost cheat-sheet

Single-agent block	$C_1 = p_{\text{in}}[NB_w + h\frac{N(N-1)}{2}] + p_{\text{out}}Na$
Supervisor routing	$C_{\text{route}}(W) = \sum_{k=1}^{W}\big[(B_s + (k{-}1)r)p_{\text{in}} + a_s p_{\text{out}}\big]$
Supervisor synthesis	$C_{\text{synth}}(W) = (B_s + Wr)p_{\text{in}} + a_y p_{\text{out}}$
Multi-agent total	$C_{\text{multi}}(W) = C_{\text{route}}(W) + W\,C_1 + C_{\text{synth}}(W)$
Multiplier	$C_{\text{multi}}(W)/C_1$ (here $1.33\times \to 5.29\times$ for $W{=}1\to4$)
Supervisor tax	$C_{\text{route}} + C_{\text{synth}}$ — pure overhead, super-linear in W
Per-layer compounding	each management layer $\approx$ doubles the coordination tax
Task latency	$L_{\text{task}} = L_{\text{route}} + \max_w L_w + L_{\text{synth}}$
Tool vs. agent	tool $= +T_{\text{schema}}$ tokens; agent $=$ + full route+synth round-trip

10 An honest word about these numbers

The token sizes here ($B_s = 1000$, $r = 1200$, $a_s = 100$) and the worker block $C_1 = \$0.0477$ are illustrative round numbers, picked so the sums are followable by hand — exactly as Module 0.0's single-agent figures were. Your real supervisor prompt, your worker result sizes, and how many workers a task touches will move every dollar figure, and the precise multipliers ($1.33\times$, $2.57\times$, …) will shift with them. The hierarchical "$\sim$2× per layer" is likewise a *rough* doubling — the exact ratio ($1.51\times$ here on tax, $1.80\times$ on call count) depends on how much fixed base each supervisor call carries versus how much accumulated context it re-reads.

What is *not* illustrative is the structure. (i) A supervisor system costs $W\,C_1$ of real work plus an irreducible routing-and-synthesis tax that grows super-linearly in W. (ii) Stacking supervisors multiplies that tax, because each layer re-reads the layer below. (iii) The supervisor sits on the latency critical path that parallel workers cannot remove. (iv) A tool is a token cost inside an existing round-trip; an agent is a whole new round-trip — so the agent-split is the categorically more expensive move. Change the constants and the dollars move; these four shapes do not. They are why the module says "default to one agent," "bound everything, cap everything," and "split only when a signal genuinely fires."

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce C_1, the multiplier table, the hierarchy comparison, the latency figure, and the tool-vs-agent table; change `B_s`, `r`, the worker constants, or the prices to explore your own system's economics.

```
# multi_agent_cost.py -- reproduces every number in "The Multi-Agent Cost Multiplier".
P_IN, P_OUT = 3e-6, 15e-6 # $ per input / output token ($3 / $15 per 1M)

```

```python
# --- single-agent ReAct block C1 (Module 0.0 shape) ---
def react_cost(B, a, h, N):
    t_in = N*B + h*(N*(N-1))//2
    t_out = N*a
    return t_in*P_IN + t_out*P_OUT

B_w, a_w, h_w, N_w = 2000, 300, 1800, 3
C1 = react_cost(B_w, a_w, h_w, N_w)
print(f"C1 = ${C1:.4f}")

# --- supervisor over W workers ---
B_s, a_s, a_y, r = 1000, 100, 300, 1200 # sup base, route out, synth out, worker result

def sup_route(W):
    return sum((B_s + (k-1)*r)*P_IN + a_s*P_OUT for k in range(1, W+1))
def sup_synth(W):
    return (B_s + W*r)*P_IN + a_y*P_OUT
def multi(W):
    route, work, synth = sup_route(W), W*C1, sup_synth(W)
    return route, work, synth, route + work + synth

for W in (1, 2, 3, 4):
    route, work, synth, tot = multi(W)
    print(f"W={W}: route=${route:.4f} work=${work:.4f} synth=${synth:.4f} "
          f"C_multi=${tot:.4f} mult={tot/C1:.2f}x")

r3, w3, s3, t3 = multi(3)
print(f"tax(W=3) = ${r3+s3:.4f} on top of ${w3:.4f} real work")

# --- hierarchical: ~2x coordination per layer (leaf work fixed at 4*C1) ---
def mgmt(W): return sup_route(W) + sup_synth(W)
leaf = 4*C1
mgmt_flat = mgmt(4)
def top_mgmt(n_sub, roll):
    t = sum((B_s + (k-1)*roll)*P_IN + a_s*P_OUT for k in range(1, n_sub+1))
    return t + (B_s + n_sub*roll)*P_IN + a_y*P_OUT
mgmt_hier = 2*mgmt(2) + top_mgmt(2, roll=2*r)
calls_flat, calls_hier = 4+1, 2*(2+1)+(2+1)
print(f"flat : calls={calls_flat} tax=${mgmt_flat:.4f} total=${leaf+mgmt_flat:.4f}")
print(f"hier : calls={calls_hier} tax=${mgmt_hier:.4f} total=${leaf+mgmt_hier:.4f}")
print(f"tax ratio hier/flat = {mgmt_hier/mgmt_flat:.2f}x calls {calls_hier/calls_flat:.2f}x")

# --- latency: supervisor on the critical path ---
L_route, L_synth, L_w = 0.8, 1.2, [3.0, 5.0, 4.0]
print(f"L_parallel = {L_route + max(L_w) + L_synth:.1f}s "
      f"L_serial = {L_route + sum(L_w) + L_synth:.1f}s "
      f"sup tax = {L_route + L_synth:.1f}s")

# --- 6th tool vs 2nd agent ---
T_schema = 120
def k_tools(k): return react_cost(B_w + max(0, k-4)*T_schema, a_w, h_w, N_w)
base4 = k_tools(4)
for k in (4, 5, 6, 7):
    print(f"1 agent {k} tools: ${k_tools(k):.4f} (+${k_tools(k)-base4:.4f})")
add2 = k_tools(6) - base4
split = multi(2)[3] - C1
print(f"add 2 tools: +${add2:.4f} split to 2 agents: +${split:.4f} "
      f"ratio = {split/add2:.0f}x")

# Expected output:
# C1 = $0.0477
# W=1: route=$0.0045 work=$0.0477 synth=$0.0111 C_multi=$0.0633 mult=1.33x
# W=2: route=$0.0126 work=$0.0954 synth=$0.0147 C_multi=$0.1227 mult=2.57x
# W=3: route=$0.0243 work=$0.1431 synth=$0.0183 C_multi=$0.1857 mult=3.89x
```

```
# W=4: route=$0.0396 work=$0.1908 synth=$0.0219 C_multi=$0.2523 mult=5.29x
# tax(W=3) = $0.0426 on top of $0.1431 real work
# flat : calls=5 tax=$0.0615 total=$0.2523
# hier : calls=9 tax=$0.0927 total=$0.2835
# tax ratio hier/flat = 1.51x calls 1.80x
# L_parallel = 7.0s L_serial = 14.0s sup tax = 2.0s
# 1 agent 4 tools: $0.0477 (+$0.0000)
# 1 agent 5 tools: $0.0488 (+$0.0011)
# 1 agent 6 tools: $0.0499 (+$0.0022)
# 1 agent 7 tools: $0.0509 (+$0.0032)
# add 2 tools: +$0.0022 split to 2 agents: +$0.0750 ratio = 35x
```

— *end of document* —